

REN- 202 - The Rhapsody Interface

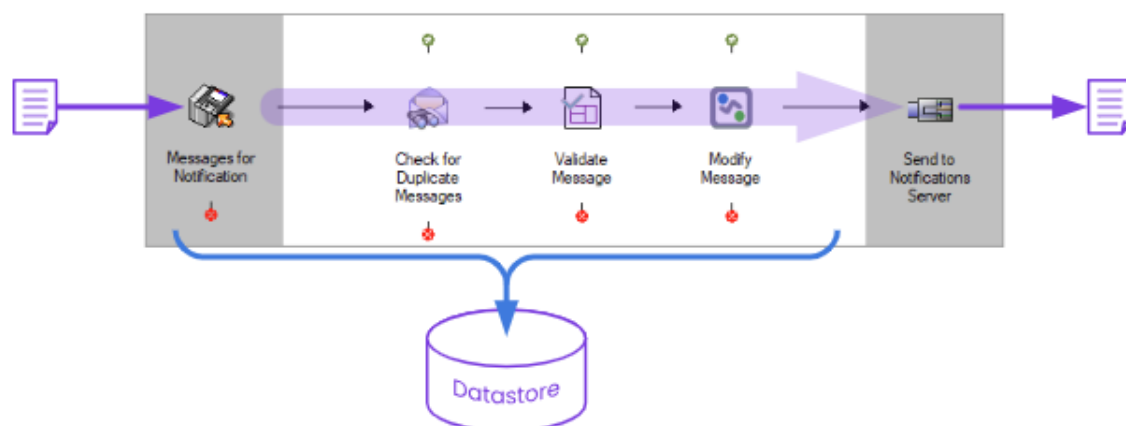
Rhapsody Processing

The Rhapsody Interface was introduced in Rhapsody Associate. This section provides an extended discussion of the interface and communications. Make sure you are familiar with the full discussion of Rhapsody communication models and basic processing information before continuing with this course. Refer to the relevant modules in the Associate courses.

Processing Model

Rhapsody processes each message individually through the input communication point, all routes and filters until it reaches the output communication point. At each stage during processing, the state of the message is saved to the datastore.

The datastore is used to guarantee message delivery and to provide a mechanism for us to review the state of previously processed messages. Where an error occurs during processing the datastore assists in correcting the error and allowing reprocessing of the message.



Communications

Each communication point provides support for a single communication protocol or protocol family and has the task on input of ensuring that the message is received, persisted to the datastore and placed onto the route processing queue. An output communication point implements the reverse.

Each communication point supports a number of processing modes (the specific modes being dependent on the protocol) and most are able to support multiple connections.

Input communication points generally operate in one of two modes, dependent on the protocol implemented;

- Polling

The communication point is responsible for polling the upstream source at regular intervals (for example, Database, Directory, FTP, SFTP).

- Server

The communication point creates a resource and passively waits for connection from a source system (for example, TCP, HTTP, Web Service Hosting).

Connection Management

Communication points have the ability to manage behavior if a connection to a remote message repository fails. This is provided by the Retry configuration items and provides support for a range of behaviors from immediate failure to infinitely attempting to recreate the connection at specified intervals.

Additional support is provided for the Out->In synchronous case to ensure that the communication point does not block indefinitely if a message is not available to the input action within a specified time period. Further detail is included in this course in the module 'Communicating with Rhapsody'.

Message Flow Management

Limitations may exist in the ability of a system to receive or process messages as rapidly as possible.

The communication point configuration model supports the ability to throttle input to and/or output from the communication point to limit the number of messages in a defined time period. Make sure you are familiar with the bursty and linear message flows as discussed in the relevant Rhapsody Associate course.

Message Flow

The Rhapsody route provides the vehicle to implement the required processing logic as a message passes through the system.

Make sure you are familiar with route operation and message flow, in particular:

- •

Receiving messages to the route queue either from a communication point or from a cross-route connection from another route.

- •

Parsing messages according to definitions attached to the route, creating a Rhapsody Messaging Toolkit message object to aid content management and extraction. This is optional and not always present.

△ If more than one definition is attached to the route, the definition to parse a message is not deterministic. For this reason, a route should only use a single definition.

- •

Filters, including an overview of their basic functionality.

- •

Persistence to the datastore for each message for each route input connection.

- •

Output

Connectors

The output from a filter may be controlled by applying a condition to its Output (default) connection. Messages are passed to the Output connector only if they match its set conditions; otherwise they will be passed to the No-Match connector. Conditions cannot be placed on the No-Match connector. If a No-Match connection is not defined for a filter, any message that would otherwise be sent to it will be sent to the Error Connector.

- •

Error

Management

If an error occurs during processing of the message by a filter, the message will be passed to the Error connector. If a connection has not been defined for that connector, Rhapsody will attempt to send the message to the Route Error Handler. If the Route Error Handler has not been configured, the message will be placed directly into the Error Queue. The Error connector provides the ability to provide special handling for errors that occur in a specific filter.

Managing Message Flow

Although each message traversing the Rhapsody engine is independent of all other messages, the environment provides support for managing special cases which otherwise would require extensive configuration;

- Managing Sets and Groups of messages
- Detecting Duplicate Messages
- Tracking and managing responses to outbound messages

Each processing element's functionality is provided by a filter. Filters typically provide support for a relatively straightforward functionality, although the JavaScript and Mapper filters can be used for more complex tasks.

Sets and Groups

It may be necessary in a configuration to ensure that messages are correctly sequenced. For example, it may be necessary that patient registration details are recorded before an admission request is received.

The message collector functionality, available to all filters, provides the ability to hold messages at input to a filter and to release them in an ordered set or simply to release them as a group.

The message collector utilizes a rule set to order the messages prior to releasing them to the filter.

Duplicate Message Detection

The Duplicate Message filter provides support for detecting duplicates based on a unique field in the messages over a defined time period.

It also provides the ability to reset detection where a re-transmission of messages is required for operational reasons.

Message Tracking

Message tracking provides support for matching of messages in an inbound stream to messages previously sent in an outbound stream without requiring extensive configuration.

Tracking is based on establishment of tracking schemes which identify a unique field in the output messages and the matching field in the response message. The schemes also determine the message retry behavior and the action to take if a matching response is not received within a specified time period.

Message Sequencing and FIFO

Each message received by Rhapsody is completely independent of any other message. There are no mechanisms for storing the state or detail of a message to apply to a subsequent message during processing, unless specifically implemented by the configuration.

Rhapsody implements a high-performance parallel processing environment, providing a number of worker threads to process messages traversing routes. By default, 10 worker threads are configured, and it is rarely necessary to modify this number. One consequence of the parallel processing model is that messages traversing a route may be processed out of sequence in the default configuration.

A number of mechanisms exist to ensure that message sequencing requirements are met:

- Constrain message concurrency for all filters to 1 (this ensures that there will never be more than one message being processed by each filter at any time)

- Enforce FIFO requirements to ensure strict message processing on the route (effectively, treat the route as single threaded)
- Group messages into a set using a message collector and then process as a set

Managing FIFO

The Rhapsody queue structure provides support for managing message ordering in the queues. Make sure you are familiar with FIFO (First In First Out) which was explained in Rhapsody Associate.

Careful route design is required to ensure that FIFO constraints do not negatively impact message flow.

Intended Use Statement

The Rhapsody Integration Engine has been designed to solve a particular range of problems. These are described in the intended use statement.

[Intended Use Statement](#)

What is Best Practice –

Rhapsody is a high performance integration engine capable of sustaining very high message loads on most hardware platforms.

Configurations may be simple, composed of only a few routes and communication points, or complex, composed of many routes and communication points.

Enterprise level implementations of Rhapsody (that is, where Rhapsody is deployed as an organization wide messaging solution) typically have in excess of 100 routes and 300 communication points. Very large sites typically have configurations with in excess of 1,000 routes and 3,000 communication points.

The goals for building a configuration remain the same, whatever the size of the site. Fundamentally, Rhapsody deals with patient data, and the ultimate goal for the configuration is to enhance (or at least have no negative effect on) patient outcomes. The configuration should therefore be:

- Error free (configuration should implement the specification correctly and be able to be validated to demonstrate correctness); zero errors is an appropriate goal.
- Maintainable (component purpose and function should be clear).
- Able to ensure that all messages should reach the correct destination in the correct form.

- Able to ensure that messages which invoke an exception in processing are identifiable and processed appropriately.
- Able to ensure that messages are processed through the engine in a timely manner (as required by the system specification).

What is Best Practice

Best Practice configuration is a broad topic which ultimately resolves to some fundamental principles which should be continuously applied during configuration and maintenance of a Rhapsody site. These principles will be encountered frequently during the Rhapsody Professional course, some directly and some indirectly. Broadly, they can be summarized as follows:

- Build Route layouts which clearly identify purpose and functionality and which are presented logically.
- Ensure that the configuration is built to process messages efficiently.
- Establish a clear naming convention for filters and connectors which identifies their function.
- Establish a clear naming convention for communication points which identifies their purpose.
- Document components using a standard template approach.
- Document any changes.
- Test each component fully before submitting the configuration, and ensure test messages and procedures are incorporated into the configuration.

Additional best practice guidelines apply to the design of a configuration; these will be discussed in a later module.

Naming Conventions

Naming conventions are focused on allowing the implementation of configurations which are essentially self-documenting; they should enable components to be labeled in a manner which is clear and indicative of purpose. This will simplify maintenance by both internal and external resources by reducing the time needed to understand the purpose of the configuration.

Naming conventions typically generate active discussion; hence it is not appropriate to direct, but rather to provide areas for discussion.

Character Set

The range of valid characters available for use in component names should be established. Key points for consideration include:

- Use of spaces in names. Spaces are preferred as word separators since they format better in Rhapsody and are easier to read.
- Valid characters; while the IDE limits the character set to the alphanumeric set (0-9, A-Z, a-z) and a few punctuation characters, it is normally preferable to limit punctuation to -_.

Component Names

Components should be named according to function and purpose. Activities which benefit from a sound naming convention include:

- Searching for a component in the **Management Console**.
- Analyzing and understanding the processing logic.

For example, it is common practice to include a flag for component type in the names of communication points such as **Finance Input DIR**, **TCP 20001 EPIC Input**. In both of these examples, the type can be used to filter the component list pages to rapidly locate the required component.

Things to consider:

- Include type flag (**Finance Input DIR**) for communication points.
- Include direction flag (**TCP RIS 1584 Out**) for communication points.
- Include source or target recipient name.
- Include type flag for filters (for example, establish a leading or trailing acronym for the filter type to enable identification of filter type in the **Management Console**).
- Location of type flags (leading or trailing, separation character).
- Casing of names (for example, camel case – see [Wikipedia\(opens in a new tab\)](#) for examples).
- Include significant configuration information; for example, port number for TCP communication points.

Including all of the above could result in a long component name. As a rule of thumb, if a name occupies more than three lines in the Route Canvas, it should be shortened to fit this recommendation. The use of spaces in a name rather than underscores is also recommended, as it aids their readability.

Direction Indicators

Where direction indicators are used in component names, they should be Rhapsody-centric. That is, Input means that Rhapsody is receiving the message, while Output means that Rhapsody is sending the message.

Prefixes and Suffixes

It may be beneficial to include a type prefix (or suffix) in the naming convention to assist in global identification of the component type; for example, a prefix of tcp could indicate a TCP communication point.

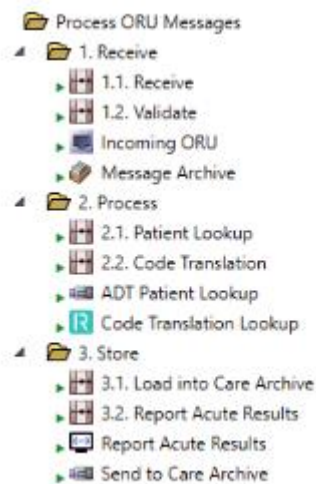
A benefit of including a type prefixes is that sorting in the Management Console or IDE then automatically sorts based on component type.

Communication Point names can be augmented with mode suffixes that indicate how it operates.

Suffix	Purpose
I	In
O	Out
IO	In->Out
OI	Out->In
E	Error out
B	Bidirectional

Sequences

Where a message will pass through a sequence of routes, the routes or folders containing the routes in the sequence can be prefixed with a number. The number ensures that the routes always appear in the same order that messages are process. As an additional benefit, the use of a sequence number prefix helps indicate to anyone else reviewing the configuration that the routes are part of a sequence.



Locker Names

Lockers are used to isolate different parts of a configuration and to prevent some users from accessing configuration or sensitive messages in parts of the Engine. Lockers should be named in a way that clearly identifies the solution or project they are part of.

Folder Names

Folders are used in the IDE and Web Management Console to provide structure and organization to the configuration. Things to consider include:

- Naming conventions which allow easy identification of folders by comparison to routes and communication points.
- High level indication of function / purpose of routes held under the folder. For example, a folder might be used to group processing logic for input from a specific host and the host system type might be included in the folder name.
- Indicator of source and or target system for messaging for the child routes, such as **PAS ADT** or **PAS to RIS**.
- Remember that folders can contain sub-folders, which allows a set of components to be grouped within a larger solution.

Route Naming

Routes are used in the IDE and Management Console to provide information on the path taken by a message as it passes through the Rhapsody engine.

- Naming Convention should allow easy identification of the route's purpose, such as *PAS ADT Feed* or *PAS to RIS*.

- You can also number routes to indicate the order in which they will process messages. For example, the source route would have the prefix 1 and the delivery route prefix 2.

Filter Names

Filters should be named according to function and purpose. The name should indicate what is happening to the message at this point in the configuration. Filters should never be left with their default name.

Naming Message Properties

A message property should be defined on the route where it is used. Although Rhapsody message properties can be passed between routes, this suggested restriction ensures that people can see where the message property came from and can locate it easily when needed. Therefore, if the property is not set on the route it is used in, it should be set using a filter that is clearly named. For example, *Set StoreToDatabase Property*.

Naming Conditional Connectors

The description for a Conditional Connector should be used to provide the condition with a succinct name that indicates its purpose. In the absence of a succinct description, the condition will be displayed on the connection; this is often long and complex.

For example, by default a condition that filters message by message type might look like this: *(ADTA36)(ADTA47)(ADTA04)(ADTA48)(ADTA27)(ADTA08)*, which is too long to view in the Route Editor. In this case, a label of ADT would likely be sufficient.

EDI Definitions

The naming convention for definitions should include the system that the EDI definition represents and the type of message being transported.

For example: *AGFA ADT HL724.s3d*.

Map Definitions

Mapping definitions describe the details of mapping one format to another format. The definition name should be self-describing: *<source system>_<format>_to_<target format>*.

For example, *EPIC ADT to RIS HL722.mdf* would indicate a mapper definition that took Epic ADT messages and translated them into HL7 version 2.2 message for an RIS system.

If there is only one mapping for a route, you could choose names for the mapping and the route that are similar to make the connection obvious.

Rhapsody Variables

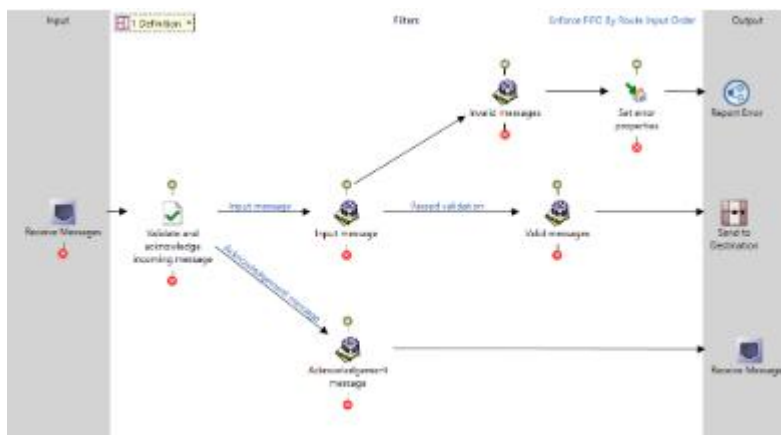
The format for Rhapsody Variables should identify the system, component, and type.

For example, RIS_TCP_Port would indicate the Port number used to connect via TCP to the RIS system.

Route Layout and Best Practice

In essence, the route design canvas of the IDE presents a map or flow chart of the high level logic required to process messages. The message flow commences with Input, on the left of the map, continues through the pathways and branching, defined in the Filters portion of the map, and terminates with the Output from the route on the right of the map. The logical flow should be from left to right. Connectors should not cross each other. The route should have the minimal number of components necessary to achieve its objectives.

The Rhapsody IDE presents a graphical environment to assist in the construction of the configuration. Routes are constructed on a graphical canvas which provides for assignment of Input and Output components and for establishing processing logic through the placement of filters and connectors.



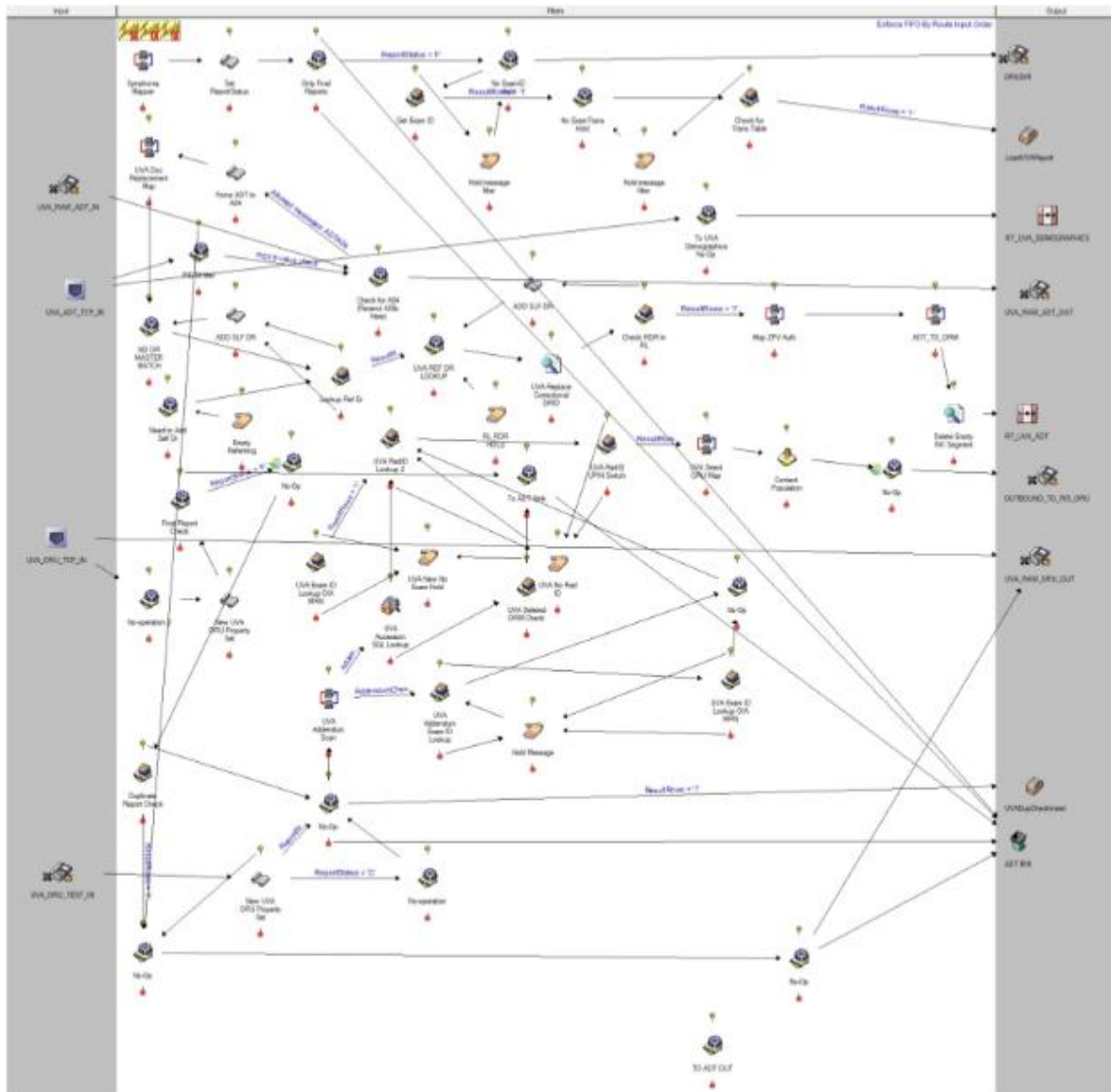
Any route may have multiple pathways, and multiple Input and / or Output components.

The free-form nature of the graphical editing environment makes it easy to add and connect components. However, there is the risk that the layout may deviate from the goal of route design, which is that the processing logic be clear and obvious.

Numerous examples exist in the wild where a route has become overly complex and difficult to understand. In both cases illustrated here, the complexity is because the configuration should be separated into multiple routes, and, in one case, the dependency upon a feedback loop should be made clear in the layout.



An overly complex Rhapsody route



Another overly complex Rhapsody route.

Guidelines

The graphic view of the Route presented in the IDE Route Canvas is a high-level summary of the processing logic applied to messages traversing the route.

Communication points

It is preferable to have a communication point instance in a single route, even if the message is to be used by multiple routes. This is because it is easy to lose track of routes likely to be affected by stopping or starting a communication point. This may be achieved by placing the communication point on a separate route and establishing a message feed to the requisite downstream routes.

This is also more efficient in terms of storage requirements. Each connection from a communication point will save a copy of the message to the datastore, so receiving once and distributing is more efficient.

In situations where one of the downstream routes is not running, messages will be placed into the route queue for the disabled route waiting for processing to resume. Careful queue management is advised in these scenarios, as large queues may have undesirable impacts on the engine, particularly at engine restart.

FIFO

Route design needs to take FIFO into account. If there is a branch in the route which would enable a message to be delayed, the design should be modified so that further messages are not delayed by the slow processing of previous messages.

Message definitions

The route framework permits one or more message definitions to be registered with the route. If required by the processing logic, the input message is parsed prior to processing by the first filter on the route, and an implicit property is added to the message containing the name of the definition used in parsing.

Caution should be exhibited where it might be deemed necessary to add multiple definitions to the route, as the definition used for parsing in this case will be indeterminate because the engine will simply pick a definition. In this scenario, it is preferable to separate the parsing step from the processing step and ensure that the correct definition is used at the parsing step.

Path management

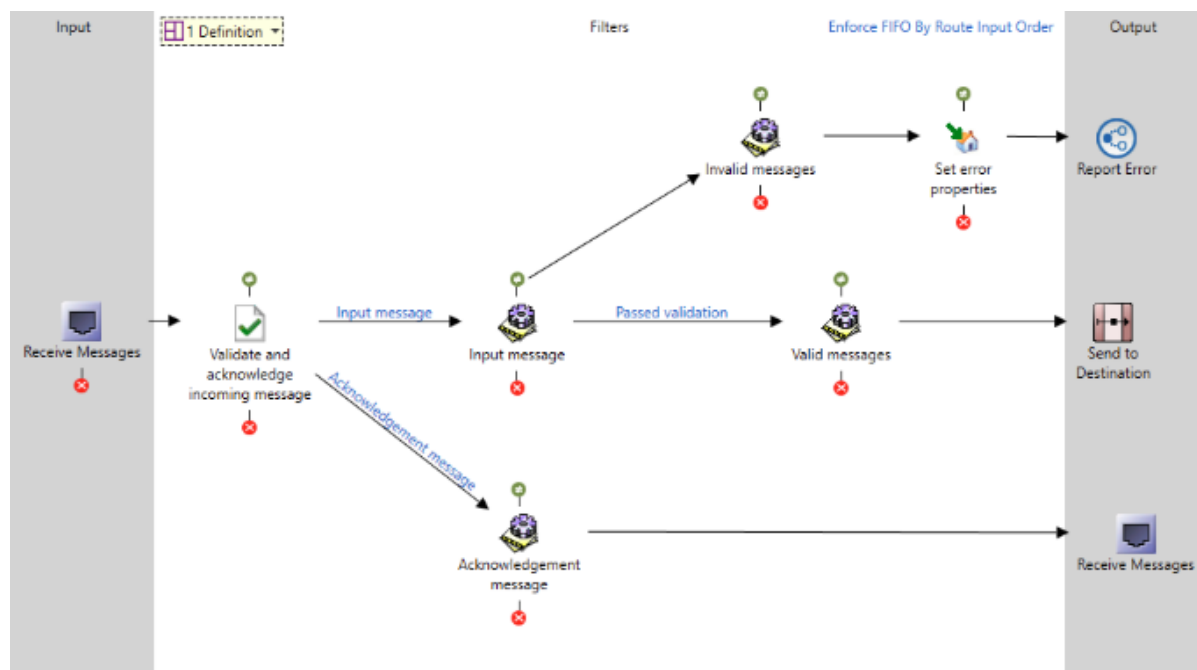
The processing logic should generally follow a path from left to right.

Paths should not cross (use No-operation filters if necessary).

Each path should generally proceed towards the output connections; that is:

- Input connections should be made from a component to the left of the filter.
- Output connections should be made to components to the right of the filter. This will be limited to some extent by connections from the No-match and Error connection points.
- No-match connections should be made above and to the right of the filter.
- Error connections should be made below and to the right of the filter.

Where it is necessary to deviate from these guidelines, ensure that filters are correctly labeled to indicate the processing path and the No-operation filters are used to keep the flow logic clear.



An example Rhapsody route.

Building a Route

A route consists of:

- One or more Input components.
- One or more processing paths consisting of connections and possibly filters.
- One or more Output components.
- Optionally, an attached message definition.
- Optionally, a registration in one or more Watchlists for monitoring.
- Properties, including startup state, route processing priority and FIFO handling.

Does a Route Need a Communication Point?

Strictly, no, although the route requires input from a source and also needs to output to a target.

A site requirement may be enforced, however, to ensure that each route has at least one communication point. That is, it is either the first or the last route in a short chain. Note that this will restrict the number of routes in a chain or place arbitrary processing requirements onto the routes. For example, it may result in routes becoming overly complex, or may artificially use TCP or other communication points in the chain to achieve the goal.

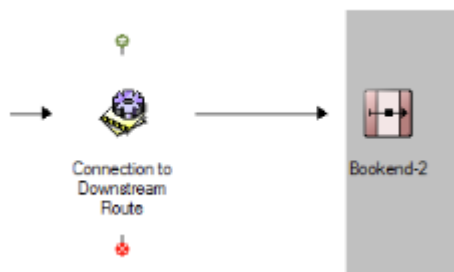
Cascading Routes

Where the route processing requirements are complex, it may be preferable to chain routes together to ensure that the clarity and purpose of each route is maintained.

In this situation, a route may be used as the input or output component of another route, and requires careful management of the connections to or from the chained route.

The preferred method of connecting two routes is as follows:

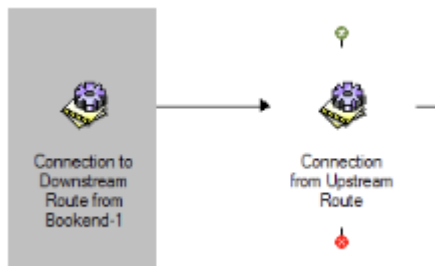
- Place a No-operation filter as the last processing element in the upstream route.
- Place a No-operation filter as the first processing element in the downstream route.
- Drag the downstream route (selected from the configuration workspace) onto the output side of the upstream route.
- Connect the No-operation filter on the upstream route to the route icon in the Output frame; note that this will place the connection into the correct location on the paired route.
- Connect the No-operation filter on the downstream route to the connection icon in the Input frame.
- Check both routes in to save the configuration.



This model is often described as providing book-ends to the routes, and generally protects the connections between the routes. It is, however, prudent to confirm the connectivity between the routes if changes are made to either route.

Note that multiple input connections may be made to the output route icon in the upstream route. Each will result in a separate input connection to the downstream route, allowing separation of processing for each path.

A route should contain no unused components as they will impact processing time if configured into the processing pathways. They will also have an impact on the use of memory by the Rhapsody engine.



A route should contain no unused components as they will impact processing time if configured into the processing pathways. They will also have an impact on the use of memory by the Rhapsody engine.

Message Parsing

If a message definition is placed on the route, message parsing will occur prior to processing by the first filter on the route; this ensures that properties defined at the route level are available to that filter and any conditional connectors downstream of the filter.

Parsing a message results in the creation of a Rhapsody Messaging Toolkit object, allowing the engine to extract values from the message and to set properties if desired. This step takes finite time and should only be undertaken as necessary.

Property Extraction

Message Properties are named values which are saved to the datastore alongside the message body. Message properties may be extracted implicitly (for example, the Java Date timestamp is extracted as the property InputTime when a message is received by a communication point), or explicitly as defined by the route logic.

While properties are a component of the message and accessible from within the configuration that processes the message, they are not normally accessible when searching from messages from the Management Console.

If it is desired to use a property in a Search condition, the property may be created as a route property and defined as Indexed. The value of an indexed property may be updated at any time during its traversal of a route; the value is not indexed until the message leaves the route. If the message traverses multiple routes, it will be indexed as it exits each route. If a message is placed onto the Error or Hold Queues, message properties will be indexed to allow their values to be searchable while the messages are in the queues.

Managing Route Functionality

In a complex configuration, it is frequently useful to separate the processing logic onto multiple routes, limiting each route to one high level function. This improves route clarity for maintenance purposes.

Error Handling

Error handling covers a wide range of topics, but the goal is to create a configuration where errors are the exception. Consequently, it is preferable to handle known errors when they occur by making use of the Error Connector output from filters. This should minimize the number of messages received by the Error Queue.

In most cases, an error which occurs during filter processing will be automatically routed to the Error Connector (there are a few filters which explicitly define error handling). If the Error Output does not have an attached connection, the message will be sent directly to the Error Queue.

Options for handling errors include:

Unhandled: the message is sent to the Error Queue.

Direct handling: logic is implemented to manage messages with known errors; the error detail is available from the message object by calling the `getErrors()` method in a JavaScript filter. Once processing is complete, it may be desirable to place the message into the Error Queue; this can be achieved by passing it to the Error Queue Filter.

Note that errors relating to filter failure (rather than processing of the message content) bypass the Error Output connection and are placed directly into the Error Queue.

Filter Concurrency

Rhapsody is a multithreaded Java application and by default uses 10 worker threads to manage all route processing. In general, the processing cycle is similar to the following:

Route worker thread becomes available.

Next route operation selected from the queue. Each operation equates to processing one message through a single filter.

Processing transaction starts.

Message processed by the filter logic.

Processing transaction completes.

Route worker thread becomes available.

This design means that it is possible for multiple messages on a route to complete processing by a filter at more or less the same time; this is the normal case unless either filter concurrency restricts processing or the route is constrained to processing in strict message order by FIFO constraints.

The filter concurrency is normally set to zero, which allows the multiple and potentially out of order processing. It may be necessary to limit concurrency in one of the following cases:

Message order is significant. Messages must process in the order of receipt.

Slow filter processing. In these cases, under moderate to high loads, it may be possible for all worker threads to be processing messages through an instance of the same filter at the same time, which will effectively block other processing. Typical scenarios for this to occur are the database filters and the Execute Process filter, which shells out to execute an external command at the OS level.

Filter Concurrency is available as a filter property for filter types likely to experience this issue. Setting the concurrency to a non-zero value limits the number of messages able to be processed through the filter instance to that value. The Database and Execute Process filters are particularly prone to needing attention to concurrency.

Queuing

In general, messages flow through the Rhapsody engine almost unhindered. Messages may queue on some components at some stages as a consequence of load, slow processing or some external factor, but queuing is generally transient.

Attention should be paid during route development to the potential for messages to queue, and the routes constructed to minimize this from occurring.

There are two significant impacts of queuing on the engine:

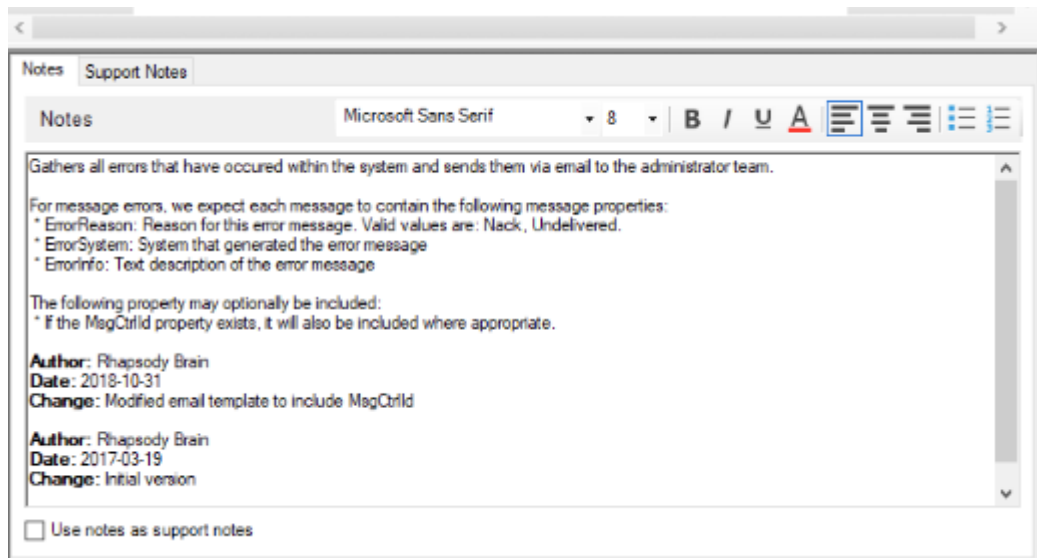
Queues are managed as memory objects. Large queues will lock memory for the queue and make it unavailable for normal processing.

When an engine restarts, the active objects are validated before processing can resume. Large queues will take time to validate and delay the resumption of normal processing by the engine.

Documenting the Configuration

The route, filter, and communication point dialogs support free text documentation to provide enhanced content for the components. Notes should be completed for all routes in your configuration, along with any communication points and filters with particularly notable configuration.

Key information should be added to the notes associated with each route that as a minimum details the functionality of the route, the change history of the route, when they were made and who was responsible for the changes. It is recommended that a standard template for documentation be established and adhered to.

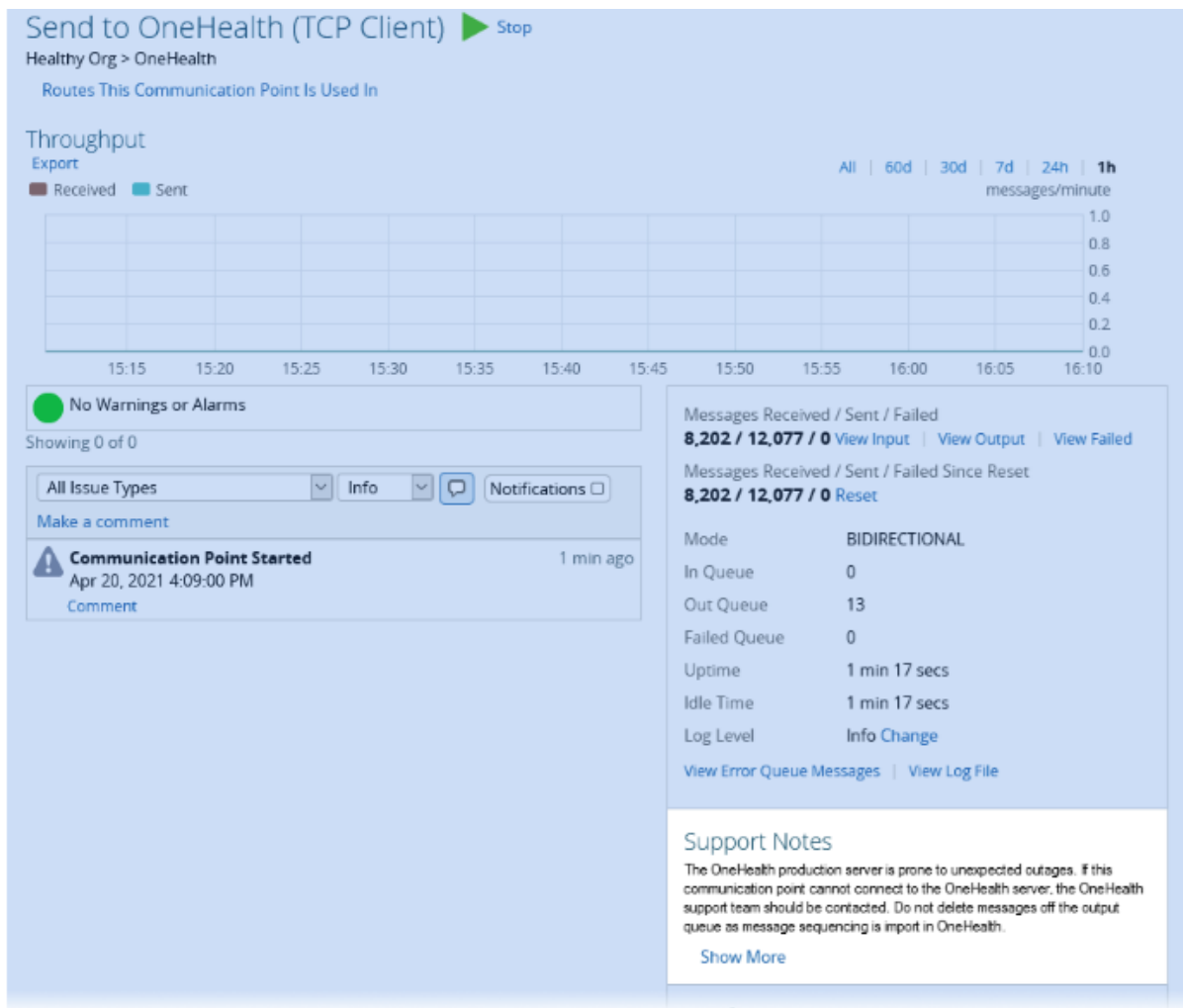


PDF documentation can be generated from the Rhapsody IDE for either the entire configuration and for a selected component. The documentation provides details of all components in the configuration. This documentation includes any notes that you have added to components in your configuration.

Support Notes

Support notes should be added to all Communication Points that connect to an external system and to important Routes to assist support staff with maintaining a solution. Support notes should include contact details for any third parties, along with any significant configuration and troubleshooting details.

Support notes can be viewed from both the Rhapsody IDE and the Management Console. This ensures the that the information can be viewed alongside the component during diagnosis of any issues that might occur.



Viewing the Support Notes for a Communication Point in the Management Console.